BUSINESS ANALYSIS PORTFOLIO · HUY HA

Smart Warehouse Inbound & Exception Management

Business Analysis case study · WMS/WCS inbound putaway — from problem framing and stakeholder analysis to traceable requirements, process models, and executed UAT.

Live demo: https://qahuy.github.io/smart-warehouse-inbound-exceptions/demo/
Repository: https://github.com/QaHuy/smart-warehouse-inbound-exceptions

---

Scope disclosure. Personal WMS/WCS portfolio case study using synthetic data. WCS behavior and device events are simulated. No interview, site observation, measured benefit, or production system is claimed.

4	10 · 10 · 5	12 / 26	14 / 14	0
business needs	FR · BR · NFR	user stories / ACs	UAT cases passed	orphaned requirements

Problem

Inbound checks are fragmented and exception visibility is poor. Barcode, SKU, quantity, duplicate-pallet, capacity, and device checks happen at different points, so nobody can say with confidence whether a pallet is safe to put away, who owns a failed movement, or whether inventory was updated before the pallet actually arrived.

Goal

A traceable pallet receipt and putaway flow with safe recovery: invalid pallets are stopped early, every WMS-to-WCS movement is traceable, exceptions are visible and bounded, and inventory changes **exactly once**, only after verified completion.

My role

Sole business analyst on a self-directed case study: problem framing, stakeholder and discovery planning, process modelling, requirements, data and interface contracts, acceptance criteria, UAT, and a clickable prototype used to validate the rules.

What I did

- Framed the problem as **4 business needs** and a fixed inbound-only scope; mapped **5 stakeholder personas** with a proposed RACI and a 45-minute interview guide.
- Modelled the assumed **As-Is** and proposed **To-Be** in **BPMN 2.0**, plus a system context, WMS–WCS sequence, and task **state machine**.
- Wrote an **SRS/FSD-lite**: 10 functional requirements, 10 business rules, 5 NFRs, and a 13-code error catalogue.
- Ordered a **12-story backlog** by dependency and operational risk; each story passes an INVEST check and has positive and negative **Given/When/Then** criteria (26 in total).
- Specified the **data and interface contracts**: 9-table SQLite schema with data dictionary, **OpenAPI 3.1** (8 operations), and 10 SQL control and metric queries.
- Validated the rules with a **clickable prototype** and **14 automated UAT cases**, linked end to end in a requirements traceability matrix (BN → FR → BR → US → UAT).

Skills demonstrated

Requirements elicitation | Stakeholder analysis & RACI | BPMN 2.0 | UML state & sequence diagrams | SRS / FSD | Business rules | User stories (INVEST) | Acceptance criteria (Given/When/Then) | Gap analysis | Requirements traceability matrix | UAT planning & execution | OpenAPI | SQL | Data dictionary | Agile backlog prioritisation | Change control

1. Context and problem

The problem statement above is a **design premise, not a measured baseline**. The consequences below are hypotheses to test in real discovery:

- Invalid pallets entering an automated putaway flow.
- Incomplete movement history between the WMS and the warehouse control system (WCS).
- Late recovery from capacity or device exceptions, with unclear ownership.
- Inventory updated before physical completion, or updated twice.

Scope

In scope	Out of scope
Barcode, SKU, quantity, and duplicate-pallet validation	Outbound, cycle count, multi-warehouse
Deterministic putaway location selection	Advanced FEFO and production optimisation
WMS movement task and simulated WCS command/events	Real ERP, PLC, robot, conveyor, or lift integration
Exception queue, bounded retry, cancellation, escalation	Enterprise authentication and production deployment
Inventory location, audit trail, and synthetic metrics	Claims about a real baseline or measured benefit

2. Stakeholders and decision rights

Five role archetypes were analysed by influence and impact. They are **proposed for the simulation**, not people who were interviewed; a 45-minute interview guide with neutral questions is ready for a real engagement.

Persona	Primary concern	Influence / impact
Inbound operator	Enter a pallet correctly and understand the next action or error.	Medium influence · high impact
Warehouse supervisor	Control the queue, retry, cancellation, and escalation.	High influence · high impact
Warehouse manager	See safe flow, exception patterns, and auditable metrics.	High influence · medium impact
Integration engineer	Preserve command/event identity, states, and traceability.	High influence · high impact
Maintenance technician	Diagnose simulated device faults without touching inventory.	Medium influence · high impact

Proposed RACI

Decision	Operator	Supervisor	Manager	Integration	Maintenance
Receipt fields and operator feedback	R	A	I	C	I
Putaway compatibility and capacity policy	C	R	A	C	I
Retry, cancel, and escalation procedure	C	A	I	C	R
Command/event contract and idempotency	I	C	I	A/R	C
Device diagnostic and recovery evidence	I	A	I	C	R
Synthetic metric meaning	I	R	A	C	I

3. Process analysis: As-Is to To-Be

The **assumed As-Is** relies on manual checks, spreadsheet lookups, and informal device coordination, so exceptions surface late and inventory is updated by hand.

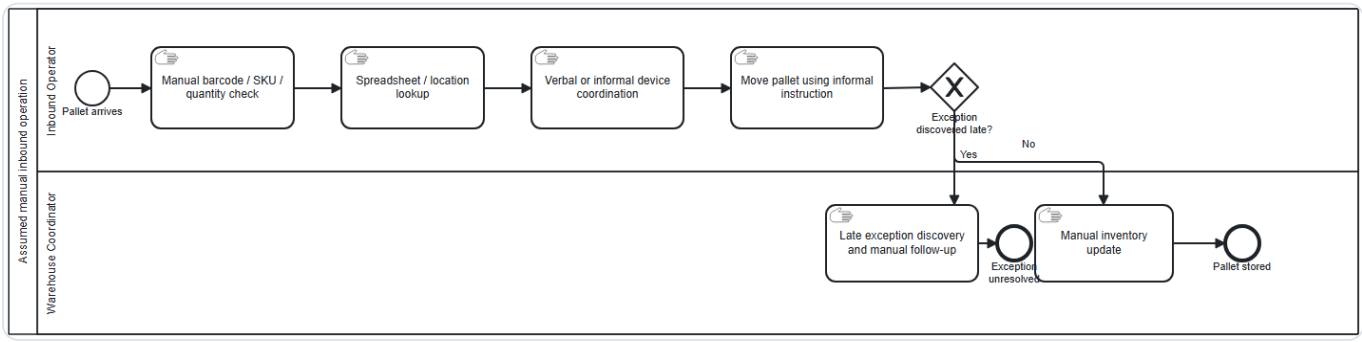


Figure 1 — Assumed As-Is inbound process (BPMN 2.0).

The **To-Be** moves every check into the WMS, makes each movement traceable, and gives every failure a visible, bounded recovery path.

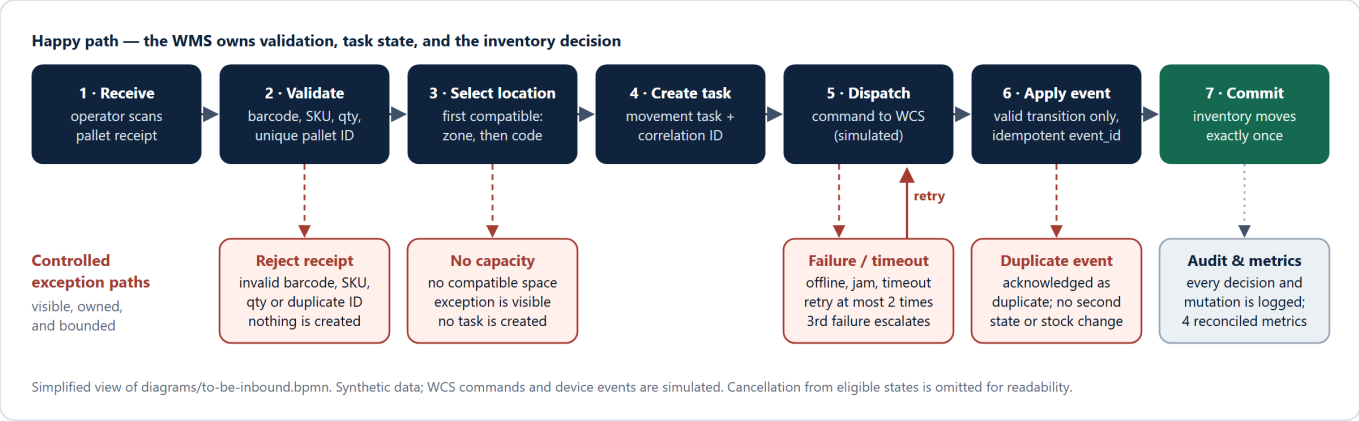


Figure 2 — To-Be flow at a glance. The full BPMN with lanes and message flows is in diagrams/to-be-inbound.bpmn.

Gap analysis

Assumed As-Is	To-Be control	Requirement
Manual barcode / SKU / quantity check	System validation at receipt with a specific error code per failure	`FR-02`, `BR-01`, `BR-02`
Duplicate pallets can enter the flow	Unique pallet identity; the original record is preserved	`FR-03`, `BR-03`
Spreadsheet location lookup	Deterministic compatible location with capacity check, or a visible capacity exception	`FR-04`, `BR-04`
Verbal or informal device coordination	Correlated WMS → WCS command and event contract with allowed state transitions	`FR-05` – `FR-07`, `BR-05`
Exceptions discovered late, ownership unclear	Exception queue; at most two retries, third failure escalates; guarded cancellation	`FR-08`, `FR-09`, `BR-08`, `BR-09`
Manual inventory update, possibly early or twice	Inventory committed once, only on a unique valid completion event	`FR-07`, `BR-06`, `BR-07`
No audit trail or reconciled metrics	Audit trail plus four metrics on a fixed dispatched-task cohort	`FR-10`, `BR-10`

4. Requirements

Business needs

ID	Business need
BN-01	Prevent invalid or duplicate pallets from entering automated putaway.
BN-02	Make each WMS-to-WCS movement traceable from dispatch to completion.
BN-03	Surface device and capacity exceptions for controlled recovery.
BN-04	Update inventory location exactly once after verified completion.

Functional requirements

ID	Requirement	Business need
FR-01	Create inbound pallet	BN-01
FR-02	Validate barcode, SKU, and quantity	BN-01
FR-03	Reject duplicate pallet ID	BN-01
FR-04	Select available compatible location deterministically	BN-01, BN-03
FR-05	Create movement task	BN-02
FR-06	Dispatch simulated WCS command	BN-02, BN-03
FR-07	Apply simulated WCS events	BN-02, BN-04
FR-08	Manage exception queue	BN-03
FR-09	Retry, cancel, and escalate	BN-03
FR-10	Expose inventory, audit, and metrics	BN-02, BN-03, BN-04

Business rules and non-functional requirements

Business rules (BR-01 to BR-10)	NFRs (NFR-01 to NFR-05)
Positive integer quantity · known active SKU · unique pallet ID	Deterministic processing
Compatible location below capacity · allowed state transitions	Auditability
Completion-gated inventory · idempotent event identity	Browser accessibility
Maximum two retries · third-failure escalation	Local operation
Metrics calculated on the dispatched-task cohort	Synthetic-data privacy

Sample user story (US-01 — Validation)

> **As an** inbound operator, **I want** immediate validation of barcode, SKU, and quantity **so that** invalid receipt data cannot enter putaway.
>
> **AC-US-01-01 (positive).** Given a new pallet ID, a valid barcode, a known active SKU, one lot, and quantity 10 — when the operator submits — then the WMS creates one pallet record and no movement task yet.
>
> **AC-US-01-02 (negative).** Given a malformed barcode, an unknown or inactive SKU, or a quantity that is blank, fractional, zero, or negative — when submitted — then the WMS returns `INVALID_BARCODE`, `UNKNOWN_OR_INACTIVE_SKU`, or `INVALID_QUANTITY` and creates neither a pallet nor a task.

The backlog orders all 12 stories by **dependency, then operational risk, then user value**: validation and identity first, event idempotency before inventory commit, and evidence before metrics. No numeric scoring is used, because no effort or value inputs exist and invented numbers would be false precision.

5. Key decisions and assumptions

ID	Decision	Rule	Why
DEC-01	Deterministic putaway	First compatible location below capacity, ordered by zone priority then location code.	Explainable and testable; no claim of route optimisation.
DEC-02	Completion-gated inventory	Inventory moves only on a valid, unique `TASK_COMPLETED` event.	Separates planned movement from physical completion; one audit point.
DEC-03	Bounded retry	Maximum two retries; the third failure escalates to a person.	Prevents silent infinite retry and makes ownership explicit.
DEC-04	UTC ISO 8601 timestamps	All simulator timestamps use UTC ISO 8601.	Unambiguous ordering across WMS, WCS, and audit.

Three assumptions are **locked but unvalidated**, each with a validation owner and a revisit trigger: one SKU and lot per pallet (`AS-01`), positive integer quantity (`AS-02`), and capacity counted in whole pallets (`AS-03`). Retry limits, cancellation rights, and RACI remain proposals until a real decision owner confirms them.

6. Exception handling and controls

Every failure has a code, a visible outcome, and an owner. A task moves through `CREATED → VALIDATED → DISPATCHED → IN_PROGRESS → COMPLETED`, with `FAILED`, `TIMED_OUT`, and `CANCELLED` reachable only through documented transitions.

Error code	Observable outcome
`INVALID_BARCODE` / `UNKNOWN_OR_INACTIVE_SKU` / `INVALID_QUANTITY`	Receipt rejected; nothing is created.
`DUPLICATE_PALLET`	Existing record unchanged; new receipt rejected.
`NO_AVAILABLE_LOCATION`	No dispatch; capacity exception visible to the supervisor.
`DEVICE_OFFLINE` / `PALLET_JAM` / `TASK_TIMEOUT`	Task fails or times out; active exception with a retry, cancel, or escalate path.
`RETRY_LIMIT_REACHED`	Third failure escalates; no third retry command is sent.
`DUPLICATE_EVENT`	Acknowledged as duplicate; no second state or inventory change.
`INVALID_TRANSITION` / `UNKNOWN_TASK`	Request rejected; state and inventory unchanged; audit retained.
`CANCELLATION_NOT_ALLOWED`	Cancellation from a terminal or undocumented state is rejected.

Data and interface contracts

- **Data model:** 9-table SQLite schema with a data dictionary, synthetic seed data, and lineage notes.
- **Interface:** OpenAPI 3.1 contract with 8 operations for receipt, task, event, retry, exception, inventory, audit, and metrics.

- **Controls:** 6 SQL integrity checks (for example, duplicate pallets, inventory count per completed task, duplicate event IDs) and 4 metric queries.

Metric	Definition (dispatched-task cohort)
Completion Rate	completed / dispatched
Exception Rate	ever failed or timed out / dispatched
First Pass Success	completed with zero retries / dispatched
Average Simulated Cycle Time	completed time minus first dispatch time

7. Validation and results

All 14 UAT cases run as automated tests (`node --test tests/uat.test.mjs`) and **pass**. The traceability matrix covers 4/4 business needs, 10/10 FRs, 10/10 business rules, and 12/12 stories with **0 orphaned requirements**.

Case	Scenario	Actual result	Status
UAT-01	Valid inbound happy path	Task completed; exactly one inventory movement	PASS
UAT-02	Malformed barcode	`INVALID_BARCODE`; no pallet created	PASS
UAT-03	Unknown or inactive SKU	`UNKNOWN_OR_INACTIVE_SKU`; no task created	PASS
UAT-04	Zero quantity	`INVALID_QUANTITY`; no pallet created	PASS
UAT-05	Duplicate pallet identity	`DUPLICATE_PALLET`; original quantity preserved	PASS
UAT-06	No compatible capacity	`NO_AVAILABLE_LOCATION` exception; no task	PASS
UAT-07	Device offline at first dispatch	`DEVICE_OFFLINE`; failure 1, retry 0, no command	PASS
UAT-08	Simulated pallet jam	`FAILED` state and `PALLET_JAM` exception	PASS
UAT-09	Simulated task timeout	`TIMED_OUT` state and `TASK_TIMEOUT` exception	PASS
UAT-10	Retry after device recovery	Redispatched as attempt 1	PASS
UAT-11	Third failure escalation	Escalated at failure 3; no third retry command	PASS
UAT-12	Cancellation state gate	Timeout cancellable; completed task rejected	PASS
UAT-13	Duplicate completion event	Flagged duplicate; still one inventory movement	PASS
UAT-14	Metric reconciliation	50%, 75%, 25%, and 120 s exactly as expected	PASS

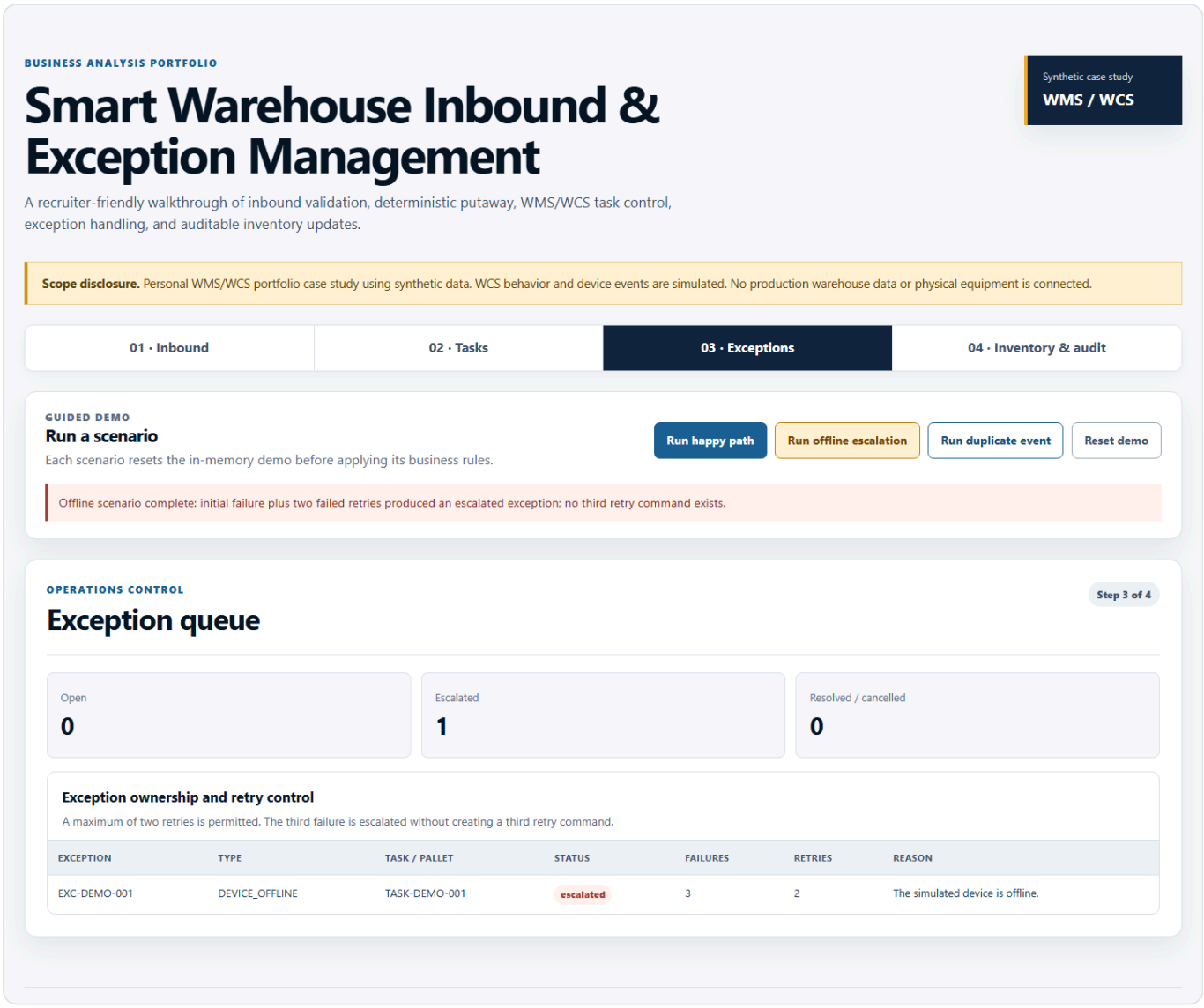


Figure 3 — Prototype: two failed retries, then escalation with no third retry command.

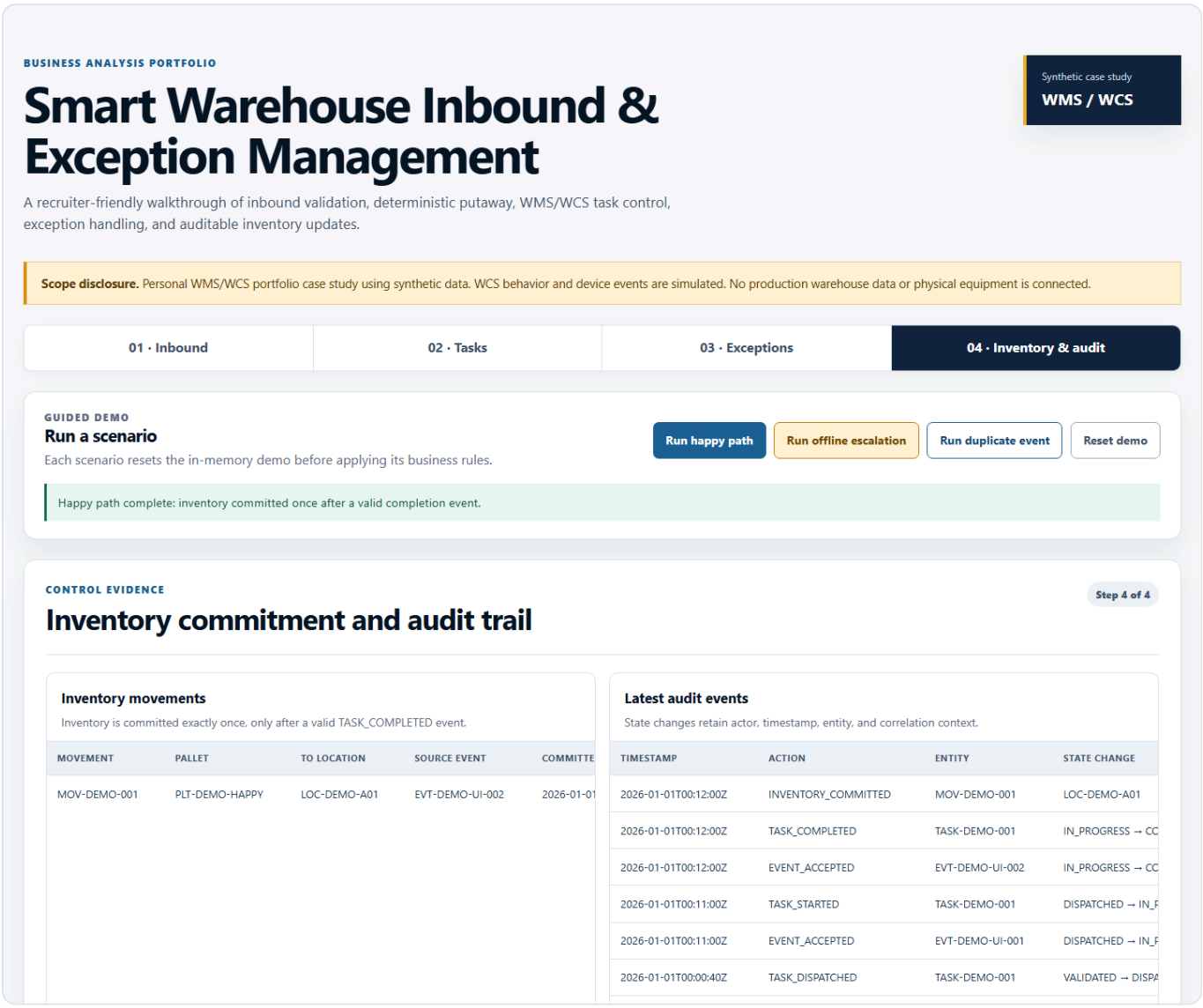


Figure 4 — Prototype: inventory committed exactly once, with the audit trail behind it.

8. Limitations and next steps

What this case study does not prove

- No warehouse user or domain expert has validated the workflow; real discovery could change rules and priorities.
- The WCS is a deterministic in-memory simulator, not a physical device or a third-party integration.
- Persistence, authentication, concurrency, and production performance are intentionally out of scope.

What I would do first in a real engagement

- Observe inbound work on site and run the interview guide with the five personas.
- Collect real exception examples and validate the As-Is / To-Be flow against them.
- Confirm data and integration contracts with the integration owner.
- Update requirements, decisions, and UAT with named decision owners and sign-off.

9. Artefact index

Artefact	Location in the repository
Case study brief, stakeholders, assumptions	`docs/01` – `docs/03`
SRS/FSD-lite, backlog, user stories	`docs/04` – `docs/06`
Traceability matrix, UAT pack, defect and change log	`docs/07` – `docs/09`
User guide and 5-minute demo script	`docs/10-user-guide-and-demo-script.md`
BPMN, context, sequence, and state diagrams	`diagrams/`
Schema, data dictionary, seed, SQL controls	`data/`
OpenAPI 3.1 contract	`api/openapi.yaml`
Prototype and automated tests	`prototype/`, `tests/`

Version 1.0 · 10 October 2026 · All examples are synthetic. Source and full artefacts: https://github.com/QaHuy/smart-warehouse-inbound-exceptions